Robot Hw 3

Andrew Bank

November 2026

1 Implementation

The planner is implemented in C++ and operates directly on the generic environment object produced by the provided parser. The supplied `main` function was left unchanged. No action names, predicate names, or domain-specific transition rules are embedded in the planner; all successor states are derived from the action schemas contained in the input environment.

1.1 State and Action Representation

A search state is represented as a set of positive grounded conditions. We use the closed-world convention: a condition not contained in the state is false. Therefore, a positive precondition is satisfied when its grounded atom is present, while a negative precondition is satisfied when the corresponding positive atom is absent. Applying an action first removes its negative effects and then inserts its positive effects. A state satisfies the goal when every grounded goal condition is present.

Each action schema is grounded by assigning environment symbols to its formal parameters and substituting those assignments into its preconditions and effects. Predicates that never appear in an action effect are static. During grounding, an action instance is discarded if one of its static preconditions is false in the initial state. Satisfied static preconditions are removed from the runtime applicability test because their values cannot change. This preprocessing is domain independent and cannot remove an action that could become applicable later.

The state container is unordered, so a deterministic key is constructed for duplicate detection by sorting the string representation of every grounded condition. Search nodes store the state, path cost, parent-node index, and the action used to reach the node. Once a goal is found, the plan is reconstructed by following parent indices back to the initial state.

1.2 Graph-Search Infrastructure

Breadth-first search uses a FIFO frontier and marks a state visited when it is inserted. This prevents duplicate frontier entries and guarantees a minimum-

action solution for unit-cost actions. A* uses a priority queue ordered by $f(s) = g(s) + h(s)$ and stores the lowest discovered g-value for each state. Stale priority-queue entries are ignored, and a state is reinserted if a lower-cost path is discovered. BFS and both A* variants share the same grounding, applicability, successor-generation, goal-test, state-key, and plan- reconstruction functions.

The planner records the plan length, grounded-action count, expanded-state count, generated-successor count, duplicate-successor count, maximum frontier size, grounding time, search time, and total planning time. These measurements are printed before the returned plan.

1.3 Completeness and Plan Quality

The reachable state graph is finite because each state is a subset of the finite set of grounded conditions. BFS graph search is therefore complete, and because every action has unit cost, it returns a plan with the minimum number of actions. A* with exact empty-delete-list cost h^+ is also complete and optimal for the positive-precondition STRIPS problems used here. Removing delete effects produces a relaxation, so its optimal solution cost cannot exceed the true remaining cost and is therefore admissible. The normalized goal-count heuristic is also admissible. If M is the largest number of goal literals that any one action can add, then any plan satisfying k currently unsatisfied goals must contain at least $\lceil k/M \rceil$ actions. Both admissible A* variants therefore retain the minimum-action guarantee.

2 Compilation and Execution

The implementation uses only the C++ standard library and requires a compiler with C++11 support. CMake version 3.16.3 or newer may be used, but no external libraries are required. From the submitted `code` directory, the CMake workflow is:

```
mkdir build
cd build
cmake ..
cmake --build . --config Release
```

On Linux, the executable is normally `build/planner`. With a multi- configuration generator such as Visual Studio, it may instead be `build/Release/planner.exe`. The planner can also be compiled directly from inside the `build` directory:

```
g++ ../src/planner.cpp -std=c++11 -O2 -o planner
```

The program takes the basename of an environment file stored in `code/envs`. For example:

```
./planner Blocks.txt
```

The search algorithm is selected with the `PLANNER_MODE` environment variable. The supported values are `bfs`, `astar-goal`, and `astar-edl`. If the variable is not defined, exact EDL A* is used. Example Linux commands are:

```
PLANNER_MODE=bfs        ./planner Blocks.txt
PLANNER_MODE=astar-goal ./planner Blocks.txt
PLANNER_MODE=astar-edl  ./planner Blocks.txt
```

The corresponding PowerShell syntax is:

```
$env:PLANNER_MODE = 'astar-edl'
.\build\Release\planner.exe Blocks.txt
```

3 Search Algorithms and Experimental Evaluation

3.1 Planner Implementations

We implemented three forward state-space graph-search planners over the supplied STRIPS representation. All actions have unit cost, so plan quality is measured by the number of actions in the returned plan.

Breadth-first search (BFS). BFS serves as the uninformed baseline. States are marked visited when they are inserted into the frontier, preventing repeated expansion of the same symbolic state. Since every action has unit cost, BFS is complete on the finite state spaces considered here and returns a minimum-length plan.

A* with goal-count heuristic. The first informed planner uses

$$h_{\text{goal}}(s) = \left\lceil \frac{|\{g \in G \mid g \notin s\}|}{M} \right\rceil, \tag{1}$$

where M is the maximum number of goal literals appearing in the positive effects of any one grounded action. A plan of n actions can add at most nM goal literals, so satisfying k currently unsatisfied goals requires at least $\lceil k/M \rceil$ actions. This makes the heuristic admissible while keeping it inexpensive and domain independent. In every benchmark domain, each action adds at most one goal literal, so $M = 1$ and the normalized value equals the raw unsatisfied-goal count.

A* with empty-delete-list heuristic. The second informed planner uses the exact empty-delete-list heuristic h^+. For each state evaluated by the outer A* search, a relaxed planning problem is constructed by removing every negative action effect while retaining action preconditions. The heuristic value is the length of the shortest plan in this relaxed problem. We solve the relaxed

problem with an inner A* search. An event-driven $h_{\max}$ propagation supplies an admissible lower bound to the inner search, but the value returned to the outer search is the exact relaxed plan length rather than the $h_{\max}$ approximation. For the positive- precondition STRIPS domains used in this assignment, h^+ is admissible because removing delete effects cannot make the relaxed problem harder than the original problem.

Domain-independent preprocessing. Before search, action schemas are grounded using the symbols in the environment. Predicates that never appear in any action effect are identified as static. A grounded action is discarded if one of its static preconditions is false in the initial state, and satisfied static preconditions are removed from its runtime applicability test. This pruning is semantics preserving because a static fact cannot change during planning. For example, it reduces the large Robot Courier instance from 6,591 unfiltered symbol assignments to 50 potentially applicable grounded actions.

3.2 Experimental Methodology

The planner was compiled as C++11 with optimization level `-O2`. Each planner/environment combination was executed three times, and the tables report the median total wall-clock planning time. Total time includes action grounding, heuristic initialization, and graph search, but excludes environment-file parsing and console output. We additionally recorded the number of expanded states, generated successors, and maximum frontier size. Timing measurements were made under WSL on the same computer for every configuration. Because sub-millisecond measurements are sensitive to system noise, expansion counts are the more stable comparison for the smallest instances.

3.3 Required Environments

Table 1 gives results for the supplied environments. The trivial BlocksEasy instance is included as a correctness check even though it is not required in the final comparison. All three planners returned plans of identical length on every problem.

The heuristic planners substantially reduce search effort on the nontrivial Blocks problems. On Blocks and Triangles, goal-count A* reduces expansions from 1,564 to 94 (94.0%), while exact EDL reduces them to 15 (99.0%). Despite this reduction, EDL takes longer than BFS because each outer state evaluation requires solving an additional relaxed planning problem. Goal-count A* provides the best runtime on this instance because its heuristic is inexpensive and still gives strong guidance.

The Fire Extinguisher domain exhibits a smaller heuristic advantage. Goal-count A* reduces expansions by 11.3% relative to BFS, while EDL reduces them by 33.5%. Exact EDL is again slower in wall-clock time. This demonstrates that the number of expanded states alone does not measure total planning cost: a

Table 1: Results on supplied environments. Time is the median of three runs and includes both grounding and search.

Environment	Planner	Grounded	Plan	Expanded	Generated	Max frontier	Time (ms)
BlocksEasy	BFS	45	1	5	18	9	0.398
BlocksEasy	A* goal count	45	1	1	5	5	0.341
BlocksEasy	A* EDL	45	1	1	5	5	0.581
Blocks	BFS	45	3	28	97	15	0.749
Blocks	A* goal count	45	3	5	26	15	0.444
Blocks	A* EDL	45	3	3	19	15	5.949
Blocks and Triangles	BFS	315	6	1564	15605	2197	152.149
Blocks and Triangles	A* goal count	315	6	94	647	237	9.897
Blocks and Triangles	A* EDL	315	6	15	140	95	507.585
Fire Extinguisher	BFS	117	21	379	2808	55	14.834
Fire Extinguisher	A* goal count	117	21	336	2494	55	12.767
Fire Extinguisher	A* EDL	117	21	252	1869	79	341.741

Table 2: Towers of Hanoi scaling results.

Disks	Planner	Grounded	Plan	Expanded	Generated	Max frontier	Time (ms)
3	BFS	72	7	25	73	8	1.668
3	A* goal count	72	7	14	41	6	1.382
3	A* EDL	72	7	13	38	4	5.971
4	BFS	126	15	67	200	16	4.815
4	A* goal count	126	15	49	146	10	4.773
4	A* EDL	126	15	45	134	8	56.554
5	BFS	200	31	237	709	32	19.574
5	A* goal count	200	31	168	503	17	14.173
5	A* EDL	200	31	159	476	16	826.239
6	BFS	297	63	675	2024	64	59.226
6	A* goal count	297	63	563	1688	31	50.865
6	A* EDL	297	63	541	1622	32	14651.600

more informed heuristic is only beneficial in runtime when the saved expansions outweigh the cost of computing it.

3.4 Towers of Hanoi Scaling Study

Towers of Hanoi was used to test the planners as problem size increases. The optimal solution for n disks has $2^n - 1$ actions; every planner returned this known optimum for all four instances. Table 2 shows the expected rapid growth in both solution length and planning effort.

EDL consistently expands the fewest Hanoi states, but its runtime scales poorly. The delete relaxation does not represent the need to move smaller disks away and later restore them: once a relaxed action establishes a fact, that fact remains true. Consequently, the relaxed problem supplies only a modest reduction in outer expansions while exact computation of h^+ becomes increasingly costly. For six disks, EDL reduces expansions by 19.9% relative to BFS but requires 14.7 seconds, compared with 59.2 milliseconds for BFS. It nevertheless

Table 3: Results on the custom Robot Courier domain.

Instance	Planner	Grounded	Plan	Expanded	Generated	Max frontier	Time (ms)
Courier	BFS	16	10	28	50	6	1.336
Courier	A* goal count	16	10	24	44	5	1.363
Courier	A* EDL	16	10	13	26	6	5.161
Courier Large	BFS	50	30	362	698	22	17.544
Courier Large	A* goal count	50	30	342	659	25	16.684
Courier Large	A* EDL	50	30	206	401	27	1066.500

remains below the assignment's 30-second planning limit.

3.5 Robot Courier Custom Domain

The Robot Courier domain tests generality beyond stacking problems. A mobile robot navigates a graph of locations, can carry at most one package, and must deliver each package to its package-specific destination. Static predicates encode object types, graph connections, and destinations; dynamic predicates encode robot/package locations, carrying state, and completed deliveries. The large instance increases the problem from four to eight locations and from two to four packages without changing the action schemas.

All planners return the same 10-action and 30-action plans for the small and large instances, respectively. On Courier Large, EDL reduces expansions by 43.1% relative to BFS, showing that the relaxed problem provides useful search guidance. Its 1.07-second runtime is nevertheless higher than the approximately 18-millisecond uninformed baseline because exact relaxed planning is performed for many outer states.

3.6 Discussion

Across all environments, heuristic search reduces the number of expanded states. The inexpensive normalized goal-count heuristic provides the best wall-clock runtime on most instances while retaining admissibility. Exact EDL is the most effective method by expansion count, but its nested relaxed searches introduce a large computational cost. The results therefore illustrate a central tradeoff in heuristic planning: heuristic accuracy must be balanced against evaluation cost.

Static-precondition pruning is important to making the EDL comparison fair. It does not alter the relaxed problem or the returned heuristic value, but avoids considering ground actions that can never become applicable. Likewise, the inner $h_{\max}$ computation only orders the exact relaxed search and does not replace h^+. Thus, the optimized EDL implementation remains faithful to the empty-delete-list definition while avoiding unnecessary work.